



Relational Database Architecture

Module: Phase 1 | Week 2 (Days 1 & 2)

1. Executive Summary & Architectural Shift

The primary objective of Week 2, Days 1 and 2, was to completely overhaul the data storage architecture. In Week 1, the pipeline outputted scraped data into Comma-Separated Values (CSV) flat files. While flat files are lightweight and easily transferable, they are wholly inadequate for enterprise Machine Learning pipelines that require high throughput, strict data validation, and concurrent access.

This phase successfully transitioned the storage layer from a static flat file to a local **Relational Database Management System (RDBMS)** utilizing SQLite. Furthermore, we decoupled the database generation from manual terminal entry by writing Python automation scripts to instantiate the database, execute Data Definition Language (DDL) schemas, and seed initial Data Manipulation Language (DML) payloads.

2. The Limitations of Flat-File Storage (CSV)

Before detailing the RDBMS implementation, it is critical to document why the deprecation of CSV was necessary for this project's scaling trajectory:

- **Lack of Data Types:** In a CSV, the number `500.00` and the text `"five hundred"` are both stored as raw strings. The Python application layer is forced to parse and type-cast every single cell, consuming massive CPU cycles.
 - **No Validation / Constraints:** A CSV cannot prevent duplicate emails or null (empty) critical values.
 - **$O(N)$ Search Complexity:** To find a specific user in a 10-million-row CSV, the Python script must read the file from line 1 to line 10,000,000 (a Full Table Scan). This results in $O(N)$ linear time complexity, which leads to `Out of Memory (OOM)` errors in ML DataLoaders.
 - **Concurrency Issues:** If two Python scripts attempt to write to the same CSV at the exact same millisecond, file corruption occurs.
-

3. RDBMS Core Principles (ACID Compliance)

By implementing an SQLite database, our storage layer now adheres to **ACID** properties, which are mandatory for financial or user-based data systems (such as our Crypto/User tracker).

1. **Atomicity:** Transactions are "all or nothing." If a script tries to insert 5 users and crashes on the 4th, the entire transaction is rolled back. No partial data is saved.
 2. **Consistency:** The database ensures that any data written strictly follows the predefined Schema rules.
 3. **Isolation:** Concurrent transactions occur independently without interfering with each other.
 4. **Durability:** Once the Python script executes `connection.commit()`, the data is permanently written to non-volatile disk memory.
-

4. Data Definition Language (DDL) & Schema Architecture

To enforce Consistency, we architected a strictly typed schema utilizing SQL DDL commands. Below is the technical breakdown of the `users` table architecture.

4.1 Schema Definition

```
CREATE TABLE IF NOT EXISTS users (  
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    first_name TEXT NOT NULL,  
    last_name TEXT NOT NULL,  
    email TEXT UNIQUE NOT NULL,  
    role TEXT DEFAULT 'customer',  
    account_balance REAL DEFAULT 0.00  
);
```

4.2 Column-Level Engineering Rationale

- `user_id INTEGER PRIMARY KEY AUTOINCREMENT`: We opted for a *Surrogate Key* rather than a *Natural Key* (like email). The database automatically assigns an auto-incrementing 64-bit integer. This ensures absolute uniqueness and immutability.

- **first_name & last_name TEXT NOT NULL** : NOT NULL is a hard constraint. If the Python application attempts to push a user without a name, the database engine physically rejects the payload, throwing an `IntegrityError` . This protects downstream ML algorithms from encountering NaN (Not a Number) exceptions.
 - **email TEXT UNIQUE NOT NULL** : The `UNIQUE` constraint prevents duplicate record creation. Under the hood, SQLite creates a secondary index specifically for the email column to enforce this rule efficiently.
 - **account_balance REAL DEFAULT 0.00** : Floating-point mathematics. By setting a `DEFAULT` , the application layer does not need to explicitly send a balance parameter for new users, reducing payload size.
-

5. Indexing & Time Complexity (Big O Notation)

The implementation of the `PRIMARY KEY` drastically alters how data is retrieved from the hard drive.

Without a Primary Key, the database must perform a **Sequential Scan** $O(N)$.

By declaring `user_id` as the Primary Key, the RDBMS automatically constructs a **B-Tree (Balanced Tree) Index**.

When Python requests `user_id = 500` , the engine does not scan 500 rows. It traverses the B-Tree, halving the search space at each node. This reduces the search time complexity to $O(\log N)$. In a table of 1 million users, finding a specific user takes roughly 20 operations instead of 1,000,000.

6. Data Manipulation Language (DML) Mechanics

Once the DDL schema established the structural blueprint, we utilized DML to seed the database.

```
INSERT INTO users (first_name, last_name, email, role, account_balance)
VALUES ('Jainish', 'Mehta', 'jainish@example.com', 'admin', 500.50);
```

Mechanism of Action:

Because we omitted `user_id` from the column list, the SQLite engine intercepts the insertion, calculates `MAX(user_id) + 1` , and generates the primary key dynamically.

7. Application Layer Integration (Python `sqlite3`)

Data pipelines must be fully automated. To bridge the gap between the application memory (Python) and the storage disk (SQLite), we utilized the `sqlite3` driver.

7.1 Connection and Cursor Objects

```
connection = sqlite3.connect(db_path)
cursor = connection.cursor()
```

- **The Connection:** Establishes a raw data stream between the Python runtime environment and the `.db` file on the OS filesystem.
- **The Cursor:** A cursor is a control structure that enables traversal over the records in a database. It acts as the "worker" that takes SQL string queries from Python, translates them into C-level instructions for the SQLite engine, and holds the resulting data in memory.

7.2 File I/O and Execution

Instead of hardcoding SQL strings into the Python file (which violates the separation of concerns), the script utilizes Python's `open()` function to read the raw `.sql` file. It then utilizes `cursor.executescript()`, which allows multiple SQL statements separated by semicolons to run synchronously.

7.3 Transaction Management (`commit` and `close`)

```
connection.commit()
connection.close()
```

SQLite uses a Write-Ahead Log (WAL) or Rollback Journal. When `executescript()` runs, changes are made in temporary RAM. If the power cuts out, the data is lost. `connection.commit()` forces the OS to flush the RAM and physically alter the magnetic/solid-state storage on the hard drive. Finally, `connection.close()` releases the file lock. If a file lock is not released, subsequent scripts will throw a `Database is Locked` operational error.

8. Security, Source Control, and Environment Variables

8.1 Virtual Environment Isolation

We initialized a new Python virtual environment (`python -m venv venv`) specific to the database module. This ensures that any future Object-Relational Mapping (ORM) libraries (like SQLAlchemy) do not pollute the global Python namespace.

8.2 Protecting Data via `.gitignore`

Databases contain Personally Identifiable Information (PII). Pushing PII to a public GitHub repository violates GDPR and standard Infosec protocols.

We appended `*.db` to the project's `.gitignore` file. This acts as a Git firewall. The source code (`build_db.py` and `Schema_1.sql`) is safely pushed to the cloud, allowing other engineers to generate the database locally, while the actual `company.db` containing data remains strictly on the local machine.

9. Code Appendix

9.1 `Schema_1.sql`

(See standard Week 2 Schema code)

9.2 `build_db.py`

(See standard Week 2 Build code)
